

S²-RAG: Decoupled Semantic and Structural Indexing for Long-Document Retrieval-Augmented Generation

Jianing Yang, Guojie Liu, Yiqi Wang^(✉), Songlei Jian^(✉), Jianfeng Zhang, Shuang Tan, and Bao Li

College of Computer Science and Technology, National University of Defense Technology, ChangSha, China
yjn@nudt.edu.cn

Abstract. Although the context windows of Large Language Models (LLMs) continue to expand, accurate question answering and multi-hop reasoning over long documents remain challenging. The core difficulty lies not only in processing more tokens, but also in accurately locating dispersed key information and establishing cross-paragraph factual connections. Traditional Retrieval-Augmented Generation (RAG) methods typically segment documents into independent text chunks and rely on dense Retrieval for recall. While effective for semantic matching, they are limited in modeling structural dependencies among dispersed evidence. GraphRAG methods support multi-hop navigation and structured retrieval, but usually depend on LLMs to construct graph structures, resulting in high indexing costs. To address these challenges, we propose **S²-RAG**, a dual-index framework with decoupled Semantic and Structural indexing for long-document Retrieval-Augmented Generation. Inspired by the human reading strategies of skimming and scanning, it employs a hierarchical summary tree for global semantic localization and an entity co-occurrence graph for local factual navigation, thereby capturing the semantic overview and structural cues of long documents in a complementary manner. In the online stage, a dual-path fusion mechanism is introduced to jointly rank and integrate candidate evidence from the semantic and structural paths, improving evidence coverage and generation quality. Experimental results on multiple long-document question answering benchmarks show that S²-RAG substantially reduces indexing cost while achieving performance comparable to or better than existing GraphRAG methods.

Keywords: Retrieval-Augmented Generation · Multi-hop Reasoning · LLM

1 Introduction

Large language models (LLMs) have rapidly improved their ability to process long contexts [8, 15]. However, the ability to accept longer inputs does not necessarily translate into strong reasoning over long documents [11, 12, 26]. For tasks

such as long-document question answering and multi-hop reasoning, models must still identify and connect relevant evidence scattered across distant parts of a document. As document length increases, irrelevant context can easily dilute attention and reduce retrieval and reasoning accuracy [11, 16, 26].

Retrieval-augmented generation (RAG) provides a natural solution to this problem by retrieving relevant evidence before answer generation [9, 10]. Conventional RAG methods typically split a document into independent chunks and rely on dense retrieval to recall relevant passages. [4, 23] Although effective for semantic matching, such flat chunk-based retrieval does not explicitly model structural dependencies across dispersed evidence, which limits its effectiveness on long-document multi-hop questions [7, 18, 27].

Graph-based retrieval methods partially address this limitation by modeling entities and their relations as graphs, enabling structured navigation across document evidence. However, existing GraphRAG methods often rely on LLMs to construct graph structures through fine-grained information extraction or relation generation. This leads to substantial indexing costs and may introduce additional structural noise [7, 18, 25].

To address these limitations, we propose S²-RAG, a dual-index framework with decoupled semantic and structural modeling for long-document retrieval-augmented generation. Inspired by the human reading strategies of *skimming* and *scanning* [17], our method decomposes long-document retrieval into a coarse-to-fine process. Specifically, skimming motivates the use of a hierarchical summary tree for rapid global semantic localization, while scanning motivates the use of an entity co-occurrence graph for local factual navigation around salient entities and clues. During retrieval, we further introduce a graph-aware ranking and dual-path fusion mechanism to integrate evidence from semantic and structural retrieval signals. Our contributions are summarized as follows:

- We propose S²-RAG, a dual-index framework that decouples semantic and structural indexing for long-document retrieval-augmented generation.
- We design a dual-path retrieval and fusion mechanism that combines global semantic localization with local structural navigation.
- Extensive experiments on multiple long-document question answering benchmarks show that S²-RAG achieves competitive or better performance than strong GraphRAG baselines while substantially reducing indexing cost.

2 Related Work

Retrieval-Augmented Generation (RAG) has become a core paradigm for mitigating knowledge gaps and factual hallucinations in large language models (LLMs) by introducing external non-parametric knowledge during generation [4]. Traditional RAG typically follows a flat chunk-index-retrieve pipeline based on dense vector similarity, but such flattened retrieval is limited in knowledge-intensive and multi-hop reasoning settings: rigid chunking may disrupt cross-sentence coreference and logical continuity, while one-dimensional semantic similarity often fails to retrieve evidence essential to the full reasoning chain [7, 20].

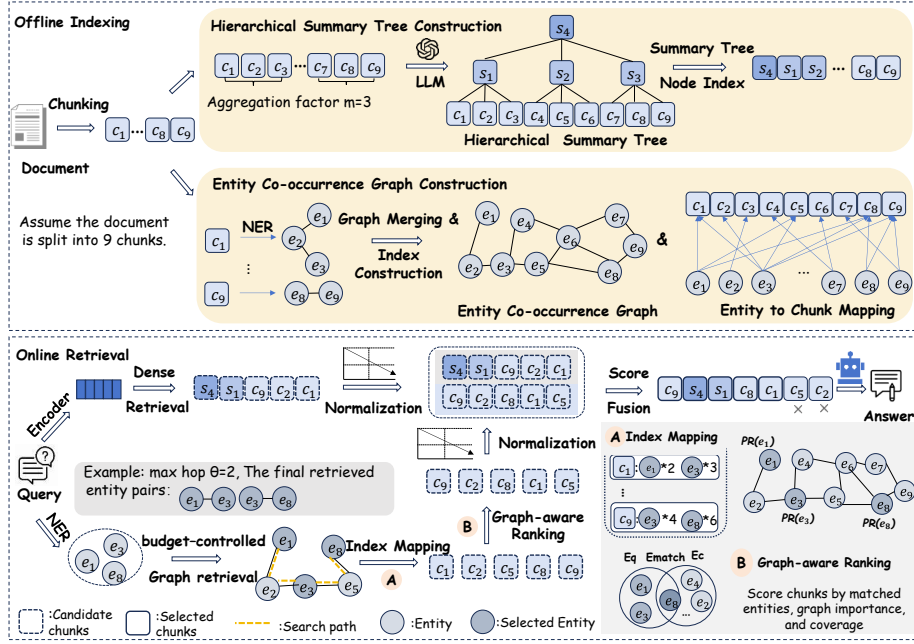


Fig. 1. Overview of S²-RAG. Offline indexing builds two complementary indices from the input document: a hierarchical summary tree and an entity co-occurrence graph with entity-to-chunk mapping. During online retrieval, dense retrieval over the summary-tree index provides semantic candidates, while budget-controlled graph retrieval produces structural candidates. The two candidate sets are normalized and fused, where (A) maps graph-retrieved entities back to text chunks and (B) applies graph-aware ranking based on matched entities, graph importance, and coverage, yielding the final evidence set for answer generation.

To address the limitations of one-shot retrieval, post-retrieval refinement methods such as Refiner reconstruct retrieved content to reduce redundant noise and improve contextual alignment [13]. Meanwhile, iterative retrieval frameworks such as IRCot and KiRAG interleave chain-of-thought reasoning with recursive multi-round retrieval to dynamically expand knowledge coverage [3, 21]. Although these methods improve evidence completeness, they remain constrained by the recall bottleneck of the initial retrieval stage or by the latency introduced by repeated inference.

Graph-based Retrieval-Augmented Generation (GraphRAG) has become an important direction for modeling structural dependencies across documents [7, 20]. Early representative frameworks such as Microsoft GraphRAG and RAPTOR extract entity relations offline and combine them with community detection or hierarchical summarization to support global knowledge understanding [2, 19]. LightRAG and HippoRAG further improve multi-hop navigation through dual-layer retrieval and personalized PageRank over knowledge graphs,

respectively [5, 6]. However, conventional GraphRAG methods rely heavily on explicit relation extraction, which incurs high token costs and may introduce cascading structural noise from local inaccuracies and global inconsistencies [1, 18]. Recent work has therefore explored lighter-weight and finer-grained topological designs. LinearRAG constructs relation-free tri-graphs using only semantic links, reducing the cost and noise of explicit relation extraction [28]. SentGraph builds hierarchical sentence-level graphs based on Rhetorical Structure Theory (RST) for denoised fine-grained evidence selection [14]. LogicRAG goes further by discarding static pre-built graphs and instead dynamically constructing and topologically sorting directed acyclic graphs (DAGs) at inference time, enabling adaptive multi-hop retrieval [1]. Together, these developments reflect a broader shift in RAG architectures toward higher efficiency and stronger logical fidelity.

3 Method

3.1 Overview and Problem Formulation

Given a long document $\mathcal{D}$ and a query q , the goal is to retrieve evidence chunks that support answer generation and then produce the answer a . S²-RAG is a dual-index framework that decouples semantic and structural modeling for long-document retrieval-augmented generation. In the offline stage, it constructs a hierarchical summary tree $\mathcal{T}$ and an entity co-occurrence graph $\mathcal{G}$; in the online stage, evidence is retrieved jointly from semantic and structural paths, and the top- K chunks form the evidence set $\mathcal{C}$ for LLM generation. The overall framework is shown in Figure 1.

Given q , S²-RAG dynamically leverages $\mathcal{T}$ and $\mathcal{G}$ to retrieve the most relevant chunk subset $\mathcal{C} \subset \mathcal{D}$:

$$\begin{aligned} \mathcal{T} &= \text{BuildTree}(\mathcal{D}) & \mathcal{G} &= \text{BuildGraph}(\mathcal{D}), \\ \mathcal{C} &= \text{Retrieval}(q, \mathcal{T}, \mathcal{G}) & a &= \text{LLM}(q, \mathcal{C}). \end{aligned}$$

Here, $\mathcal{T}$ supports global semantic localization, while $\mathcal{G}$ supports local factual navigation. Their retrieval signals are fused in a unified ranking space to improve evidence coverage and ranking quality.

3.2 Hierarchical Summary Tree Construction

The hierarchical summary tree $\mathcal{T}$ serves as the semantic index. The input document $\mathcal{D}$ is first segmented into overlapping base chunks $L_0 = \{c_1, c_2, \dots, c_n\}$, where c_i denotes the i -th chunk. Overlapping segmentation alleviates semantic truncation caused by hard chunk boundaries.

A fixed aggregation factor m is then used to recursively build higher-level nodes in a bottom-up manner. Given the node set at layer h , denoted by L_h , every consecutive group of m adjacent nodes is summarized by an LLM into one parent node, producing L_{h+1} . This process continues until the current layer size falls below a preset threshold, yielding the summary tree $\mathcal{T} = \{L_0, L_1, \dots, L_H\}$,

where L_h denotes the node set at layer h and H is the tree depth. This tree provides multi-granularity semantic representations from raw chunks to high-level summaries, enabling coarse-to-fine semantic localization.

3.3 Entity Co-occurrence Graph Construction

The entity co-occurrence graph $\mathcal{G}$ serves as the structural index. Given the base chunk sequence $\{c_1, c_2, \dots, c_n\}$, each chunk is split into sentences, from which named entities and key terms are extracted using named entity recognition and part-of-speech tagging, forming the node set V .

For any two entities e_i and e_j , an undirected edge is added if they co-occur in the same sentence. Let $f(e_i)$ denote the occurrence frequency of e_i , $f(e_i, e_j)$ the co-occurrence frequency of (e_i, e_j) , and N_{co} the total number of co-occurrence events. The edge weight is defined as

$$w_{ij} = \max \left(\log \frac{f(e_i, e_j) \cdot N_{\text{co}}}{f(e_i)f(e_j)}, 0 \right). \quad (1)$$

Only edges with $w_{ij} > 0$ are retained, resulting in a weighted graph $\mathcal{G} = (V, E, W)$.

To connect graph nodes back to text, an inverted index from entities to chunks is further built: $\mathcal{B}(e) = \{c \mid e \text{ appears in } c\}$. This index allows the system to trace graph-relevant entities back to supporting text chunks.

3.4 Dual-path Evidence Retrieval

Given a query q , candidate evidence is retrieved along both semantic and structural paths.

For the semantic path, q is encoded by an embedding model and matched against the node embedding index of $\mathcal{T}$, yielding the semantic candidate set $\mathcal{C}_{\text{sem}}$ and the raw semantic score $s_{\text{sem}}(c, q)$ for each candidate chunk c .

For the structural path, query entities $E_q = \{e_1, e_2, \dots, e_m\}$ are first extracted. The entity graph $\mathcal{G}$ is then searched for valid entity pairs satisfying the hop constraint $\text{dist}(e_i, e_j) \leq h$, where $\text{dist}(\cdot, \cdot)$ denotes shortest-path distance and h is the maximum hop threshold. For each valid pair (e_i, e_j) , the corresponding candidate set is constructed as $\mathcal{C}_{ij} = \mathcal{B}(e_i) \cap \mathcal{B}(e_j)$. The union over all valid pairs forms the structural candidate set $\mathcal{C}_{\text{graph}} = \bigcup_{(e_i, e_j) \in \mathcal{P}_q} \mathcal{C}_{ij}$, where $\mathcal{P}_q$ denotes the valid-pair set.

If $|\mathcal{C}_{\text{graph}}|$ exceeds a predefined budget B , the hop threshold h is progressively tightened until the candidate size falls within budget. If no reliable query entities are extracted or no valid entity pairs exist, the system falls back to semantic-only retrieval.

3.5 Graph-aware Ranking and Score Fusion

After obtaining $\mathcal{C}_{\text{sem}}$ and $\mathcal{C}_{\text{graph}}$, structural-path candidates are re-ranked and then fused with semantic-path candidates at the score level.

For a structural candidate chunk $c \in \mathcal{C}_{\text{graph}}$, let E_c denote the entities contained in c and E_q the query entities. On the query-relevant subgraph, each entity is assigned an importance score $\pi(e)$ using PageRank. The base structural score is

$$s_{\text{base}}(c | q) = \sum_{e \in E_q \cap E_c} \text{tf}(e, c) \cdot \pi(e), \quad (2)$$

where $\text{tf}(e, c)$ is the frequency of entity e in chunk c .

Let $M(c | q) = E_q \cap E_c$ denote the matched query entities in c . The coverage term and average importance term are defined as $\text{Cov}(c | q) = |M(c | q)|/|E_q|$ and $\text{Imp}(c | q) = \frac{1}{|M(c | q)|} \sum_{e \in M(c | q)} \pi(e)$, respectively. The final structural score is

$$s_{\text{graph}}(c | q) = s_{\text{base}}(c | q) \cdot \text{Cov}(c | q) \cdot \text{Imp}(c | q). \quad (3)$$

Since $s_{\text{sem}}(c | q)$ and $s_{\text{graph}}(c | q)$ lie in different numerical spaces, candidates from each path are normalized by rank:

$$\hat{s}_p(c | q) = 1 - \frac{r_p(c | q) - 1}{N_p - 1}, \quad (4)$$

where $r_p(c | q)$ and N_p denote the rank of c and the total number of candidates under path $p \in \{\text{sem}, \text{graph}\}$, respectively. The final fused score is

$$s(c | q) = \alpha \hat{s}_{\text{sem}}(c | q) + (1 - \alpha) \hat{s}_{\text{graph}}(c | q), \quad (5)$$

where $\alpha \in [0, 1]$ is selected on the validation set. Candidate chunks are ranked by $s(c | q)$, and the top- K are returned as the final evidence set $\mathcal{C}$. We have summarized the specific implementation details of S²-RAG into the pseudocode presented in algorithm 1.

4 Experiments

In this section, we conduct systematic experiments to evaluate the effectiveness and efficiency of S²-RAG for long-document question answering. The experiments are designed to answer the following research questions:

- **RQ1:** Can S²-RAG achieve better question answering performance than existing RAG and GraphRAG methods?
- **RQ2:** Does S²-RAG provide advantages in indexing cost?
- **RQ3:** How much do the hierarchical summary tree, entity co-occurrence graph, and score fusion module each contribute to the final performance?
- **RQ4:** How does the fusion hyperparameter α affect the system performance?

4.1 Experimental Setup

Datasets and evaluation metrics. We evaluate S²-RAG on long-document question answering benchmarks constructed from NovelQA [22] and InfiniteBench

Algorithm 1 S²-RAG Retrieval Pipeline

Require: Document D , query q , maximum hop h , candidate budget B , fusion weight α , top- K

Ensure: Evidence set $\mathcal{C}$ and answer a

- 1: Build hierarchical summary tree $\mathcal{T}$ from D
- 2: Build entity co-occurrence graph $\mathcal{G}$ and inverted index $\mathcal{B}$
- 3: Retrieve semantic candidates $\mathcal{C}_{\text{sem}}$ from $\mathcal{T}$ using q
- 4: Extract query entities E_q from q
- 5: **if** $E_q \neq \emptyset$ **then**
- 6: Retrieve graph candidates $\mathcal{C}_{\text{graph}}$ from $\mathcal{G}$ with hop limit h
- 7: **if** $|\mathcal{C}_{\text{graph}}| > B$ **then**
- 8: Shrink h until the candidate budget is satisfied
- 9: **end if**
- 10: Compute topology-aware scores for $\mathcal{C}_{\text{graph}}$
- 11: **else**
- 12: $\mathcal{C}_{\text{graph}} \leftarrow \emptyset$
- 13: **end if**
- 14: Rank-normalize scores from both paths
- 15: Fuse two paths with weight α
- 16: Select top- K chunks as evidence set $\mathcal{C}$
- 17: Generate answer $a = \text{LLM}(q, \mathcal{C})$
- 18: **return** $\mathcal{C}, a$

Table 1. Statistics of the evaluation datasets.

Dataset	# Samples	# Questions	Avg. Length (tokens)
NovelQA	89	2305	212,736
InfiniteChoice	58	229	188,415
InfiniteQA	69	351	198,074

[24]. Specifically, we use NovelQA together with two English subsets from InfiniteBench, namely InfiniteChoice and InfiniteQA. As shown in Table 1, all evaluation datasets contain extremely long contexts, with average sample lengths around 200k tokens. For NovelQA and InfiniteChoice, we use Accuracy as the evaluation metric. For InfiniteQA, which requires free-form answer generation, we adopt ROUGE-L.

Baselines. We compare S²-RAG against representative baselines from three retrieval paradigms: flat chunk-based retrieval (Vanilla RAG), hierarchical semantic retrieval (RAPTOR) [19], and graph-based retrieval approaches, including GGraphRAG [2], LGraphRAG [2], LightRAG [5], and HippoRAG [6]. This setup allows us to examine the proposed method against systems with substantially different indexing and retrieval strategies.

Implementation details. To evaluate the generality of the proposed framework, experiments are conducted with two open-source instruction-tuned backbones, Qwen2.5-7B-Instruct and Llama3.1-8B-Instruct. We use BGE-M3 for dense re-

Table 2. Main experimental results across three complex QA datasets. **Bold** indicates the best performance, and underline indicates the second best. The evaluation metric is Accuracy for NovelQA and InfiniteChoice, and ROUGE-L for InfiniteQA.

Method	Qwen2.5-7B-Instruct			Llama-8B-Instruct		
	NovelQA	InfiniteChoice	InfiniteQA	NovelQA	InfiniteChoice	InfiniteQA
Vanilla RAG	43.82	39.30	14.10	32.97	33.19	8.01
RAPTOR	38.44	43.67	11.65	42.21	40.17	9.86
GGraphRAG	34.06	36.68	5.22	36.01	34.06	3.62
LGraphRAG	39.09	37.55	9.63	48.34	43.23	7.33
LightRAG	39.87	38.43	10.32	47.74	42.79	9.01
HippoRAG	49.85	<u>50.66</u>	<u>14.23</u>	49.63	<u>49.78</u>	<u>13.67</u>
S ² RAG (Ours)	<u>49.76</u>	56.77	18.05	<u>48.50</u>	52.84	15.37

trieval. In the indexing stage, MiMo-V2-Flash is used for LLM-dependent pre-processing, including graph construction in GraphRAG-style baselines and hierarchical summary generation in summary-based methods such as RAPTOR and S²-RAG. For answer generation, experiments with both Qwen2.5-7B-Instruct and Llama3.1-8B-Instruct are conducted on a single NVIDIA RTX 4090 GPU with 48GB memory.

4.2 Main Results

Table 2 reports the results of different methods on the three benchmark datasets with two backbone models. In general, S²-RAG achieves the best or near-best performance across both backbones, demonstrating that the combination of semantic and structural indices effectively improves the organization of evidence for answering long-document questions. Specifically, on Qwen2.5-7B-Instruct, S²-RAG achieves the best performance on InfiniteChoice and InfiniteQA, and is only slightly below the best-performing baseline on NovelQA. On Llama3-8B-Instruct, S²-RAG again delivers the best results on InfiniteChoice and InfiniteQA, while remaining competitive on NovelQA. In particular, on InfiniteQA, S²-RAG consistently outperforms strong graph-based baselines, indicating a clear advantage on tasks that require stronger cross-paragraph evidence aggregation and multi-hop reasoning.

From the perspective of cross-model consistency, S²-RAG maintains relatively stable performance across both backbones, whereas several GraphRAG baselines exhibit larger performance fluctuations. This suggests that S²-RAG not only provides strong overall effectiveness, but also demonstrates better robustness across different language model backbones.

4.3 Cost Analysis

To evaluate indexing efficiency, we compare the token consumption of different methods during the graph construction stage on the NovelQA dataset. We use

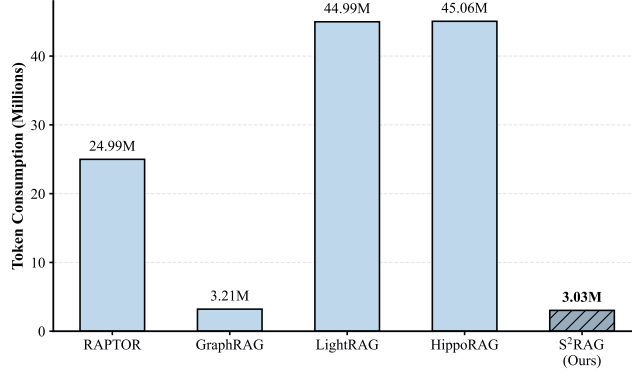


Fig. 2. Comparison of token consumption during offline index construction across different methods.

token consumption rather than wall-clock time as the primary efficiency metric, since latency measurements may be noisy due to network instability when invoking remote models. The results are shown in Figure 2. We observe that S²-RAG substantially reduces indexing cost while maintaining competitive or superior performance.

Compared with existing GraphRAG methods, S²-RAG requires significantly fewer tokens during offline index construction. In particular, its construction cost is much lower than methods that rely heavily on LLM-based relation extraction. Although RAPTOR has a comparable cost profile, S²-RAG achieves stronger overall performance in the main experiments.

These results indicate that the performance gains of S²-RAG do not come from increasing indexing complexity. Instead, by assigning global semantic abstraction to the hierarchical summary tree and local factual navigation to the lightweight entity co-occurrence graph, our method achieves a better balance between effectiveness and efficiency.

4.4 Ablation Study

To analyze the contribution of each component, we conduct ablation experiments by removing the hierarchical summary tree, the entity co-occurrence graph, and the score fusion module, respectively. The results are shown in Table 3.

The full model consistently outperforms all ablated variants across all datasets, demonstrating that each component contributes positively to the final performance. Removing either the hierarchical summary tree or the entity co-occurrence graph leads to clear performance degradation, suggesting that semantic and structural information are complementary for long-document retrieval.

Among all variants, removing the score fusion module causes the most severe performance drop. This shows that dual-path candidate generation alone is insufficient to guarantee strong evidence quality; effective ranking and coordina-

Table 3. Ablation study of S^2 RAG on three long-document QA datasets. The full model is shown in the first row, and each ablated variant reports its performance together with the absolute change relative to the full model. Red indicates performance drops. The evaluation metric is Accuracy for NovelQA and InfiniteChoice, and ROUGE-L for InfiniteQA.

Method	Dataset		
	NovelQA Acc.	InfiniteChoice Acc.	InfiniteQA R-L
S^2 RAG (Ours)	49.76	56.77	18.05
w/o Graph	43.37 ($\downarrow$ 6.39)	47.91 ($\downarrow$ 8.86)	13.43 ($\downarrow$ 4.62)
w/o Tree	43.12 ($\downarrow$ 6.64)	41.05 ($\downarrow$ 15.72)	11.53 ($\downarrow$ 6.52)
w/o Scoring & Fusion	40.69 ($\downarrow$ 9.07)	40.61 ($\downarrow$ 16.16)	13.71 ($\downarrow$ 4.34)

Table 4. Sensitivity analysis of the fusion weight α on Qwen2.5-7B-Instruct. The best result in each row is shown in **bold**.

Dataset	α			
	0.6	0.7	0.8	0.9
InfiniteChoice	53.28	56.77	54.59	53.71
NovelQA	47.85	49.76	49.03	48.97

tion between the semantic and structural paths are crucial for the final retrieval outcome.

4.5 Sensitivity Analysis

We further analyze the effect of the fusion hyperparameter α on system performance. Based on Qwen2.5-7B-Instruct, we evaluate different values of α on InfiniteChoice and NovelQA, and present the results in Figure 4.

Overall, the model performance exhibits a relatively stable unimodal trend as α varies, and reaches its optimum around $\alpha = 0.7$. This suggests that, under the current setup, assigning slightly more weight to the semantic path achieves a better balance between global semantic coverage and local structural constraints, thereby leading to stronger retrieval performance.

5 Conclusion

We presented S^2 -RAG, a dual-index framework for long-document retrieval-augmented generation that decouples semantic and structural indexing. Inspired by *skimming* and *scanning*, it combines a hierarchical summary tree for global

semantic localization with an entity co-occurrence graph for local factual navigation, together with graph-aware ranking and dual-path score fusion.

Experiments on multiple long-document question answering benchmarks show that S²-RAG substantially reduces indexing cost while achieving competitive or better performance than strong GraphRAG baselines.

References

1. Chen, S., Zhou, C., Yuan, Z., Zhang, Q., Cui, Z., Chen, H., Xiao, Y., Cao, J., Huang, X.: You don’t need pre-built graphs for rag: Retrieval augmented generation with adaptive reasoning structures. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. vol. 40, pp. 30270–30278 (2026)
2. Edge, D., Trinh, H., Cheng, N., Bradley, J., Chao, A., Mody, A., Truitt, S., Metropolitansky, D., Ness, R.O., Larson, J.: From local to global: A graph rag approach to query-focused summarization. *arXiv preprint arXiv:2404.16130* (2024)
3. Fang, J., Meng, Z., Macdonald, C.: Kirag: Knowledge-driven iterative retriever for enhancing retrieval-augmented generation. In: *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. pp. 18969–18985 (2025)
4. Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, H., Wang, H., et al.: Retrieval-augmented generation for large language models: A survey. *arXiv preprint arXiv:2312.10997* **2**(1), 32 (2023)
5. Guo, Z., Xia, L., Yu, Y., Ao, T., Huang, C.: LightRAG: Simple and fast retrieval-augmented generation. In: *Findings of the Association for Computational Linguistics: EMNLP 2025*. pp. 10746–10761 (2025)
6. Gutiérrez, B.J., Shu, Y., Gu, Y., Yasunaga, M., Su, Y.: Hipporag: Neurobiologically inspired long-term memory for large language models. *Advances in neural information processing systems* **37**, 59532–59569 (2024)
7. Han, H., Wang, Y., Shomer, H., Guo, K., Ding, J., Lei, Y., Halappanavar, M., Rossi, R.A., Mukherjee, S., Tang, X., et al.: Retrieval-augmented generation with graphs (graphrag). *arXiv preprint arXiv:2501.00309* (2024)
8. Hong, K., Troynikov, A., Huber, J.: Context rot: How increasing input tokens impacts llm performance. URL <https://research.trychroma.com/context-rot>, retrieved October **20**, 2025 (2025)
9. Karpukhin, V., Oguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: *Proceedings of the 2020 conference on empirical methods in natural language processing (EMNLP)*. pp. 6769–6781 (2020)
10. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive nlp tasks. *Advances in neural information processing systems* **33**, 9459–9474 (2020)
11. Li, J., Wang, M., Zheng, Z., Zhang, M.: Loogle: Can long-context language models understand long contexts? In: *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. pp. 16304–16333 (2024)
12. Li, T., Zhang, G., Do, Q.D., Yue, X., Chen, W.: Long-context llms struggle with long in-context learning. *arXiv preprint arXiv:2404.02060* (2024)

13. Li, Z., Hu, X., Liu, A., Zheng, K., Huang, S., Xiong, H.: Refiner: restructure retrieved content efficiently to advance question-answering capabilities. In: Findings of the Association for Computational Linguistics: EMNLP 2024. pp. 8548–8572 (2024)
14. Liang, J., Zhou, P., Zhou, W., Qing, W., Zhao, Q., Wang, Z., Song, Q., Li, X.: Sentgraph: Hierarchical sentence graph for multi-hop retrieval-augmented question answering. arXiv preprint arXiv:2601.03014 (2026)
15. Liu, J., Zhu, D., Bai, Z., He, Y., Liao, H., Que, H., Wang, Z., Zhang, C., Zhang, G., Zhang, J., et al.: A comprehensive survey on long context language modeling. arXiv preprint arXiv:2503.17407 (2025)
16. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Bevilacqua, M., Petroni, F., Liang, P.: Lost in the middle: How language models use long contexts. *Transactions of the association for computational linguistics* **12**, 157–173 (2024)
17. Maxwell, M.J.: Skimming and scanning improvement: The needs, assumptions and knowledge base. *Journal of Reading Behavior* **5**(1), 47–59 (1972)
18. Peng, B., Zhu, Y., Liu, Y., Bo, X., Shi, H., Hong, C., Zhang, Y., Tang, S.: Graph retrieval-augmented generation: A survey. *ACM Transactions on Information Systems* **44**(2), 1–52 (2025)
19. Sarthi, P., Abdullah, S., Tuli, A., Khanna, S., Goldie, A., Manning, C.D.: Raptor: Recursive abstractive processing for tree-organized retrieval. In: The Twelfth International Conference on Learning Representations (2024)
20. Sun, J., Xu, C., Tang, L., Wang, S., Lin, C., Gong, Y., Ni, L.M., Shum, H.Y., Guo, J.: Think-on-graph: Deep and responsible reasoning of large language model on knowledge graph. arXiv preprint arXiv:2307.07697 (2023)
21. Trivedi, H., Balasubramanian, N., Khot, T., Sabharwal, A.: Interleaving retrieval with chain-of-thought reasoning for knowledge-intensive multi-step questions. In: Proceedings of the 61st annual meeting of the association for computational linguistics (volume 1: long papers). pp. 10014–10037 (2023)
22. Wang, C., Ning, R., Pan, B., Wu, T., Guo, Q., Deng, C., Bao, G., Hu, X., Zhang, Z., Wang, Q., et al.: Novelqa: Benchmarking question answering on documents exceeding 200k tokens. arXiv preprint arXiv:2403.12766 (2024)
23. Zhang, Q., Xiang, Z., Xiao, Y., Wang, L., Li, J., Wang, X., Su, J.: Faithfulrag: Fact-level conflict modeling for context-faithful retrieval-augmented generation. In: Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers). pp. 21863–21882 (2025)
24. Zhang, X., Chen, Y., Hu, S., Xu, Z., Chen, J., Hao, M.K., Han, X., Thai, Z.L., Wang, S., Liu, Z., et al.: inftybench: Extending long context evaluation beyond 100k tokens. arXiv preprint arXiv:2402.13718 (2024)
25. Zhao, Y., Zhu, J., Guo, Y., He, K., Li, X.: E²graphrag: Streamlining graph-based rag for high efficiency and effectiveness. arXiv preprint arXiv:2505.24226 (2025)
26. Zhou, Y., Liu, H., Chen, Z., Tian, Y., Chen, B.: Gsm-: How do your llms behave over infinitely increasing reasoning complexity and context length? In: International Conference on Machine Learning. pp. 78933–78983. PMLR (2025)
27. Zhou, Y., Su, Y., Sun, Y., Wang, S., Wang, T., He, R., Zhang, Y., Liang, S., Liu, X., Ma, Y., et al.: In-depth analysis of graph-based rag in a unified framework. arXiv preprint arXiv:2503.04338 (2025)
28. Zhuang, L., Chen, S., Xiao, Y., Zhou, H., Zhang, Y., Chen, H., Zhang, Q., Huang, X.: Linearrag: Linear graph retrieval augmented generation on large-scale corpora. arXiv preprint arXiv:2510.10114 (2025)